

ACUCOBOL-GT Appendices

Version 9.0.1

Books

[Appendix I: Library Routines](#) > [I.1 General Syntax and Library List](#) >

S\$IO

The S\$IO routine provides an interface to the sequential file handler. Calls to the routine require an operation code and a variable number of parameters, depending on the operation called. The return code is set automatically after the call. The external variable "F-ERRNO" is set according to any errors found. "F-ERRNO" may not be reset on entry to S\$IO, and should be checked only if S\$IO returns an error condition.

Note: File locking should be applied whenever SEEK operations are used.

Usage

```
CALL "S$IO"  
    USING OP-CODE, PARAMETERS
```

Parameters

OP-CODE Numeric parameter

Specifies the file handling routine to be performed. The following table lists each op-code and its corresponding operation. The operations are detailed below:

Code	Operation
1	S-OPEN-FUNCTION
2	S-CLOSE-FUNCTION
3	S-MAKE-FUNCTION
4	S-READ-FUNCTION
5	S-WRITE-FUNCTION
6	S-REWRITE-FUNCTION
7	S-SEEK-FUNCTION

PARAMETERS variable depending on op-code

The remaining parameters vary depending on the operation selected. They provide information and hold results for the operations specified. Parameters may be omitted from those operations that do not require them. The parameters are detailed in the section titled "Description" below.

Description

All parameters passed to S\$IO are passed by reference. This applies even to parameters that are integer values in the corresponding file handling routines.

Except for the MAKE function, S\$IO automatically terminates PIC X parameters with a LOW-VALUES byte.

You do not have to specify SYNC for level 01 or level 77 parameters because they are automatically synchronized by ACUCOBOL-GT.

The "filesys.def" COPY file contains many useful definitions for use with S\$IO. It contains definitions for the S\$IO op-codes as well as the "F-ERRNO" error values. It also includes many useful pre-declared variables that are of the proper type and usage.

Note: The behavior of this routine is affected by the setting of the FILENAME_SPACES configuration variable that may or may not allow spaces in a file name. See the documentation on [FILENAME_SPACES](#) in Appendix H for information about the terminating character for path names.

The runtime configuration variables FILE_PREFIX and FILE_SUFFIX are ignored by the S\$IO routine.

OP-CODES and PARAMETERS

[S-OPEN-FUNCTION \(op-code 1\)](#)

[S-CLOSE-FUNCTION \(op-code 2\)](#)

[S-MAKE-FUNCTION \(op-code 3\)](#)

[S-READ-FUNCTION \(op-code 4\)](#)

[S-WRITE-FUNCTION \(op-code 5\)](#)

[S-REWRITE-FUNCTION \(op-code 6\)](#)

[S-SEEK-FUNCTION \(op-code 7\)](#)

S-OPEN-FUNCTION (op-code 1)

This routine opens an existing sequential file. If it is successful, the value in RETURN-CODE should be moved to a data item that is USAGE HANDLE. This data item is passed as the open file handle to the other file handling routines. If it fails, RETURN-CODE is set to a NULL value.

The S-OPEN-FUNCTION routine has four required parameters, and three optional parameters: *name*, *mode*, *resize*, *type*, *blocking*, *padding*, *pipe_name*.

Name is the name of the file to open. It need not be null-terminated.

Mode is one of the following values (defined in "filesys.def"):

Finput open for input only

Foutput open for output only

Fio open for input and output

Fextend open for output only (same as Foutput)

"Foutput" does not delete the current file (this behavior differs from the OPEN OUTPUT statement in COBOL).

Note: This routine only opens files that already exist. If the file does not exist, the routine fails, even when opening with mode "Foutput".

Mode may furthermore have one of the following flags added to it to indicate file locking options (defined in "filesys.def"):

Fread_lock locks file against other updaters

Fwrite_lock locks file against all others

Fmass_update locks file against all others (same as Fwrite_lock)

If "Fmass_update" is used, the file system is directed to emphasize speed of updates over file security.

Additionally, "Fmulti_lock" may be added to *mode* to request that more than one record lock be maintained in the file by this process. If this option is not specified, then any I/O operation on the file will first release any currently

locked record. This results in only one record lock being set in the file at any time. When this option is used, locked records are released only when the file is closed or when the UNLOCK routine is called.

Recsize is the maximum size of each record. This must be known at open time.

Type is the type of sequential file, and is one of the following values (defined in "filesys.def"):

S-FIXED	fixed record binary sequential file
S-VAR-COUNT	variable record length binary sequential file
S-LINE	line sequential file
S-PRINT	line sequential file

Blocking is the size of a block, in bytes. This parameter is optional and defaults to 0, meaning that a block is the size of a record.

Padding is the value of the pad character for filling short blocks. This parameter is optional and defaults to 0, meaning that any short blocks are padded with a binary 0 value.

Pipe-name is the name of the pipe to open instead of a file. This parameter is optional. It only has an effect on UNIX machines.

S-CLOSE-FUNCTION (op-code 2)

This routine closes an open file. It also removes currently held locks on the file. S-CLOSE-FUNCTION has only one parameter, *f*, a file handle returned by the S-OPEN-FUNCTION. For some file systems, it is possible that S-CLOSE-FUNCTION will write additional records that had been previously buffered by the system. For this reason, it is possible that a "disk full" condition can occur.

S-MAKE-FUNCTION (op-code 3)

This routine creates a new sequential file. It will overwrite any existing file of the same name. However, it will not overwrite a file that is currently in use. If the file is in use, the error E_FILE_LOCKED is returned.

This routine does not automatically terminate its parameters with LOW-VALUES. You must ensure that its parameters are correctly terminated.

The S-MAKE-FUNCTION routine takes two parameters: *name* and *l_parms*.

Name contains the name of the file. The name need not be null-terminated.

L-parms is a string that describes various logical characteristics of the file. The string consists of three numeric fields separated by commas. The string must be null-terminated. This parameter is optional. If it is not specified, the values are not known to the runtime. The fields are as follows:

1. *Maximum record size*. This is the size in bytes of the largest record to be placed in the file. This can range from 1 to 67,108,864.
2. The *file type* (accepted values are the same as described in S-OPEN-FUNCTION). This must be a single byte containing a binary value that indicates the type of the file.
3. *Block size*. This is the size of a block of records.

S-READ-FUNCTION (op-code 4)

This routine reads the next record in the sequence of records. The S-READ-FUNCTION routine has two parameters, *f* and *record*.

F must be a valid file handle returned by S-OPEN-FUNCTION.

Record points to the area to hold the record read.

If S-READ-FUNCTION succeeds, RETURN-CODE is set to the size of the record read, plus one. If it fails, RETURN-CODE is set to zero. However, if the function fails due to the record being locked, the file pointer is set to the locked record.

Records read in a file open for input only are not locked. Furthermore, most file systems do not block the reading of locked records in a file open for input (this feature depends on the host file system - not all support it). Records read from a file open for I/O are automatically locked unless the external variable "f-no-lock" is set to a non-zero value, in which case they are treated in the same manner as files open for input.

S-WRITE-FUNCTION (op-code 5)

This routine adds a new record to the named file. It has four parameters: *f*, *record*, *size*, and *cr_cntrl*.

F must be a valid file handle returned by S-OPEN-FUNCTION.

Record points to the record to add.

Size is the size of the record. If size is zero, then the maximum record size for the file is used.

Cr_cntrl is the number of lines to advance before writing the new record. This is only valid for print files.

The S-WRITE-FUNCTION routine does not change the current file position.

S-REWRITE-FUNCTION (op-code 6)

This routine replaces an existing record in the file. It has three parameters: *f*, *record*, and *size*.

F must be a valid file handle returned by S-OPEN-FUNCTION.

Record points to the new record to place in the file.

Size is the size of the record. It may be zero to indicate the maximum record size for the file. The size of the new record need not match the size of the existing record.

The S-REWRITE-FUNCTION routine does not affect the file position.

S-SEEK-FUNCTION (op-code 7)

This routine changes the current position of the file for subsequent READs.

The only time the runtime supports a seek in a sequential file is 1) to find the last record in order to read the file backwards, and 2) after each READ to seek to the previous record. *These are the only uses of SEEK supported by this library.* The file must be locked to ensure that other users of the file don't add to it.

S-SEEK-FUNCTION has three parameters: *f*, *offset* and *mode*.

F must be a valid file handle returned by OPEN.

Offset is the number of bytes to move from the current file position. Offset is limited to a 32-bit value.

Mode is a flag that specifies the position from which the offset is measured. A value of zero sets the position at the beginning of the file, regardless of the current position. A value of one sets the position to the current position. A value of two sets the position to the end of the file. Note that offset can be negative value.

Micro Focus

[Contact Micro Focus](#)

Please share your comments on this manual
or on any extend product documentation with the
[extend documentation team](#).